Smart App PH Build Plan (MVP)

Webserbisyo · Version 1.1 · October 12, 2026 · Companion to **Smart App PH System Paper (MVP Edition) v1.1**
This plan turns the system paper into work, in six phases. Each phase is a set of small checkpoints: one prompt, one result, one test, one commit. A checkpoint is done only when its "Done when" list passes on a real phone. Section numbers in brackets point to the system paper.

How every checkpoint runs

1. Claude (chat) writes one prompt file for the checkpoint into *docs/prompts/*, with the goal, files to touch, rules and the "Done when" list.
2. Claude Code or Antigravity runs it. It starts with a short plan, builds only what the checkpoint says, and finishes with *pnpm check* green.
3. The founder tests the "Done when" list with Juan's Cafe (food) and Tala Apparel (retail).
4. Commit with a clear message, push, and Vercel builds preview links to test on a phone.
5. New ideas that come up go into the roadmap list, not into the current checkpoint.

Who does what:

- **Claude Code:** repo, packages, config, database, server logic, tests.
- **Antigravity:** screens, checked in its browser.
- **Claude chat:** prompts, reviews, decisions.

Before phase 1: confirm these decisions

#	Decision	Recommendation
1	Two apps: Next.js storefront and Vite dashboard	Yes. The dashboard goes to iOS and Android later with Capacitor (printers, POS); separating now avoids a move later
2	Three shared packages: ui, core, api	Yes. Half of the first build's six; every kind of code has one fixed place [8.3]
3	Approvals and subscriptions by SQL	Yes. Saved snippets in the Supabase SQL editor [Appendix A]
4	Order types at launch: walk-in, pickup, delivery	Yes. Dine-in table QR and shipping right after launch
5	Members and stamps in the launch version	Yes. They are the selling point

#	Decision	Recommendation
6	Demo data in SQL inside Supabase	Yes. A *demo_reset* function run from the SQL editor, no AI tokens [Appendix B]
7	Reuse proven code from the first repo	Yes, as copy-and-trim reference: tokens and fonts, money and order rules, image compression, order engine SQL, checkout parts, the dashboard shell and module registry. Never copy whole folders
8	Platform domain	Pick one short domain now; it is needed for email (Resend), the dashboard (*app.domain*) and the shop links

Where code goes (keep this open while building)

Folder	Holds	Never holds
apps/storefront	Customer screens and routes (Next.js)	Database calls written by hand, business rules
apps/dashboard	Owner screens, router, layout shell, one folder per feature with its module.ts	Database calls written by hand, business rules
packages/ui	Design tokens, fonts, icons, shadcn components, shared patterns	Data fetching, Supabase
packages/core	Zod schemas and types, money, order rules, open hours, vocab, feature switches	React, Supabase
packages/api	Supabase clients, queries, mutations, TanStack Query hooks, uploads, generated DB types, mock data	Screens
supabase	Migrations (tables, RLS, SQL functions), seed.sql, demo.sql, Edge Functions	Edits to applied migrations

Phase 1: Repo and context

Goal: a clean repo where every AI session starts from the same rules and the same plan.

Checkpoints

1. **Create the repo** (GitHub, private) as a pnpm workspace with Turborepo: *apps/storefront*, *apps/dashboard*, *packages/ui*, *packages/core*, *packages/api*, *supabase*, *docs*. Branches: *main* for releases, *dev* for work.
2. **Write the context files:**
    - *CLAUDE.md*: short rules, the folder map and the commands
    - *docs/system-paper.md*: the system paper in Markdown
    - *docs/decisions.md*: one line per decision
    - *docs/stack-versions.md*: exact versions and security notes
    - *docs/prompts/*
3. **Write the contracts first:** the Zod schemas for the main shapes (shop, item, variant, cart line, order payload, order, payment, customer, stamp rule) in *packages/core*, and the list of database functions with their inputs and outputs [7.4]. Screens and the database are both built against these shapes.
4. **Copy reference code** from the first repo into a folder that is not compiled (*reference/*): tokens.css, money and time helpers, order rules (next actions), image compression, the order engine SQL, the dashboard shell and module registry. Agents copy from it when the matching checkpoint comes.
5. **Import rules in the lint config:**
    - only *packages/api* may import @supabase
    - apps may not import each other
    - *packages/core* may not import React
    - dashboard features may not import each other

CLAUDE.md rules (short version)

- Do only the current checkpoint. List the files you will touch first.
- Put code where the folder map says. If something has no clear place, stop and ask.
- Finish with *pnpm check* green (typecheck, lint, tests).
- Never read or print .env.local. No new packages without asking.
- No *any*, no type casts to silence errors, no @ts-ignore.
- Never compare the shop type in code; use feature switches and vocab words.
- Database changes only as new migration files; RLS, grants and policies in the same file.
- Money in centavos, Manila time for display, every visible text in one words file.

Done when

- The repo exists with both branches, the folder layout and the context files.
- The Zod contracts compile.
- A new AI session can explain the MVP scope and where code goes from CLAUDE.md alone.

Phase 2: Foundations

Goal: both apps run locally and on Vercel preview links, with the final look and both layouts in place, before any feature.

Checkpoints

1. **Install the stack** at exact versions:
   - storefront: Next.js 16.3.8 or newer patch, React 19.3
   - dashboard: Vite 8, React 19.3, TanStack Router, TanStack Query 5
   - shared: TypeScript 6.0.3, Zod 4, Tailwind CSS 4 (CSS-first), shadcn/ui, Tabler icons, Vitest, ESLint 10, pnpm, Turborepo

   Add the *pnpm check* script that runs on the whole repo.
2. **Look and tokens** in *packages/ui*:
   - tokens.css (navy, logo blue, marigold for celebrations, member green, neutral greys, light and dark)
   - Plus Jakarta Sans
   - shop colour variables for the 60/30/10 storefront theme

   Prove one styled button renders the same in both apps before going on.
3. **Two layouts with empty pages:**
   - storefront: bottom bar (Home, Menu, Orders, Account) and cart bar
   - dashboard: sidebar on desktop, bottom bar on phone, top bar with breadcrumbs, and a module registry so each feature adds its own routes and menu entry

   No page scrolls sideways at 390 px.
4. **Supabase:**
   - create the project (Free while building)
   - install the Supabase CLI, link it
   - add *.env.example* files with the URL and publishable key names for each app
5. **Vercel:** two projects from the same repo (storefront and dashboard); every push to *dev* gets preview links.
6. **Resend:** create the account, verify the domain, and set it as the Supabase Auth email sender (SMTP).

Done when

- *pnpm dev* starts both apps with their layouts.
- Both Vercel previews open on a phone.
- Light and dark both look right.
- *pnpm check* is green.

Phase 3: Authentication and access

Goal: the right person reaches the right screens, and the database enforces it.

Checkpoints

1. **First migration:** tenants, profiles, tenant_members, shop_settings, with RLS, grants and policies, the sign-up trigger for profiles, the *create_my_shop* function, and a test for each rule.
2. **Owner auth (dashboard):**
    - sign up with email and password, then the 6-digit email code (Resend)
    - login, logout, forgot password
3. **Shop setup and approval:**
    - setup screen: type, name, link name, with a slug check
    - the shop is created as pending, and the owner sees "Waiting for approval"
    - the founder runs the approve SQL, and the owner lands on the dashboard
4. **Guards:**
    - dashboard: route guards in the router read the signed-in owner and the shop status; pending and suspended shops get their own screens
    - storefront: *proxy.ts* refreshes the customer session on every request
    - server code checks the user with *getClaims()*
5. **Customer auth (storefront):**
    - email code sign-in, the session kept per shop
    - a customer is never treated as an owner

Done when

- A new owner goes from sign-up to the dashboard after one SQL approval.
- A logged-out visitor to the dashboard is sent to login.
- Owner A cannot read owner B's shop, by screen or by direct query (test passes).
- A customer can sign in with an email code on a storefront.

Phase 4: Front end with mock data

Goal: every launch screen built in the final look, working on typed mock data, so the flows can be clicked through before the database is wired.

Mock data approach

- **In code:** *packages/api/mocks* holds two shops (Juan's Cafe, Tala Apparel) typed with the Zod contracts from phase 1: products, variants, add-ons, about 20 customers, about 30 orders in every status. Screens read them through the same hooks they will use later, so swapping to real data in phase 5 changes only *packages/api*, not the screens.
- **In the database (phase 5):** *seed.sql* for the shops and products, and *demo_reset(shop_slug)* for customers, members, orders and stamps relative to today. Run from the Supabase SQL editor, so no AI tokens are used for test data [Appendix B].

Component research (light)

Before building, list the shadcn components and blocks each screen uses. Install them once into *packages/ui* with the shadcn CLI, then build. Expected set:

- sheet/drawer (item sheet, cart), tabs, card, badge, button, input, select
- radio group (order type, payment), dialog, sonner toasts
- skeleton, empty state, table (reports, customers), chart (sales), avatar
- switch, calendar (pickup date)

Checkpoints

1. **Storefront browse:** shop home, menu with search and categories, item sheet (variants, add-ons, quantity, note), cart with the floating cart bar.
2. **Checkout and receipt:**
    - order type, details, payment choice, privacy line
    - the receipt page with the order number, QR code, live status timeline, Upload proof, Send to Messenger, Save receipt, Join for stamps
3. **Member pages:** sign-in sheet, member card with stamps, my orders, saved address, install guide.
4. **Dashboard orders:**
    - the board with the New, Preparing, Ready, Out and Done columns (tabs on phone), one next-step button per card from the order rules in *packages/core*, the chime
    - order detail with the proof viewer, cancel and print slip
5. **Dashboard products:**
    - list with search, sold-out and visibility switches
    - product editor with photos (compressed to WebP on upload), variants with price and stock, add-on groups
    - categories
6. **Dashboard home, customers and reports:**
    - today's numbers, to-verify and waiting chips, members card, in progress list, best sellers
    - customers list with members marked
    - reports for today, 7 and 30 days, with a CSV button
7. **Settings:**
    - shop look (logo, cover, name, 60/30/10 colours with presets and a phone preview)
    - business info, hours and closed days, order types and delivery zones
    - payment accounts with QR upload, Messenger page, stamp card rule

Done when

- Every launch screen in section 4.1 of the paper can be clicked through on a phone with mock data, for both shops.
- Food shows Menu words and retail shows Products words.
- Every list has loading, empty and error states.
- No page scrolls sideways.

Phase 5: Back end, database and connections

Goal: replace mock data with Supabase, one area at a time, with the logic in the database

where money, stock and stamps are involved. Screens don't change; only *packages/api* and *supabase* do.

Checkpoints

1. **Settings and catalog tables** [7.3]:
   - tables: business_hours, closures, payment_accounts, delivery_zones, categories, items, item_photos, item_variants, add-on tables
   - RLS write rules for the owner
   - storage buckets *shop-media* (public) and *payment-proofs* (private) with folder rules

   Connect the settings and products screens.
2. **Storefront read:** *get_storefront(slug)* with open status, members count and everything the menu needs. Server-render the shop home and menu. A web app manifest and icons per shop so each shop installs as its own app.
3. **Order engine:**
   - tables orders, order_items, order_events, order_counters, payments, customers, consents
   - functions *quote_order* and *place_order*: prices from the database, stock reserved, gap-free numbers, token, client key for retries
   - *submit_payment_proof* with a duplicate reference check, *get_order*
   - Turnstile + rate limit on place_order
4. **Owner order handling:**
   - *review_payment*, *record_payment*, *advance_order*, *cancel_order* (returns stock and stamps)
   - Realtime on orders for the board, the receipt page and the chime
   - pg_cron job *expire_unpaid_orders*
5. **Members and stamps:**
   - tables loyalty_rules, loyalty_ledger, customer_addresses
   - *join_as_member* (links by mobile or email, sets member_since), *get_my_account*
   - stamps added on completed paid orders and reversed on cancel, reward redeemed in place_order
6. **Home and reports:** *get_dashboard_today*, *get_member_insights*, *get_sales_report*, CSV export of orders.
7. **Demo data:** *seed.sql* and *demo_reset(shop_slug)* with the safety rules (is_demo shops only, demo markers, not callable by clients). Run it from the SQL editor.

Done when

- All ten acceptance tests in section 13 of the paper pass on a phone against the real database.
- The cross-shop test passes.
- *demo_reset* refreshes both demo shops in one SQL call.

Phase 6: Performance, deployment and launch

Goal: fast on mobile data, safe in production, first shop live.

Checkpoints

1. **Performance:**
    - storefront pages server-rendered with small client parts
    - images through *next/image* with sizes
    - dashboard screens loaded per route, heavy parts (charts) loaded only when needed
    - fonts self-hosted
    - check both apps on a mid-range phone on mobile data: the shop home should feel instant and the menu should scroll smoothly
2. **PWA polish:**
    - per-shop icons generated from the logo
    - install prompt after the first order
    - iPhone install steps
    - an offline page
    - the dashboard installable as an app too, until the native app exists
3. **Production setup:**
    - Supabase Pro (daily backups), Vercel Pro for both projects
    - the domain on Vercel: *app.domain* for the dashboard, *domain/shop* for storefronts
    - Resend sending from the domain
    - production environment variables, all migrations pushed to the production project
    - approve-SQL snippets saved
4. **Legal and trust:**
    - privacy notice page per shop, consent line at checkout
    - "ORDER SLIP · NOT AN OFFICIAL RECEIPT" on slips
    - terms for shops (monthly ₱599, data ownership)
5. **Launch check:**
    - run the ten acceptance tests on production with a test shop
    - onboard the first real shop
    - print its shop QR
    - watch its first day of orders

Done when

- A real shop takes real orders for a full day with no lost order and every payment verified in the dashboard.

After launch: the order of next features

One at a time, each as new folders, new files and new migrations:

1. New-order email to the owner, QR scanner in the dashboard
2. Dine-in table QR
3. Shipping by region
4. Promotions and vouchers
5. Staff accounts (cashier, kitchen/packer)

6. Advance orders with a calendar and deposits
7. Push notifications and announcements
8. Native dashboard app with Capacitor: iOS and Android builds of *apps/dashboard*, Bluetooth receipt printers
9. POS, cash shifts and kitchen display
10. Full reports and Google Sheets
11. Admin screen and billing automation
12. Theme editor, custom domains, PayMongo, Meta Pixel

Daily working routine

1. Morning: run *select demo_reset('juans-cafe'); select demo_reset('tala-apparel');* in the Supabase SQL editor.
2. Pick the next checkpoint. Ask Claude (chat) for its prompt file.
3. Paste the one-line instruction into Claude Code or Antigravity.
4. Test the "Done when" list on your phone with both shops.
5. Commit and push. Write any new idea into the after-launch list.

Prompt template for every checkpoint

```
# Checkpoint P{phase}.{n}: {name}
Read CLAUDE.md and docs/system-paper.md section {x} first.
Goal: {one sentence}
Build only: {list}
Do not build: {anything nearby that is out of scope}
Where the code goes: {apps/..., packages/ui|core|api, supabase/...}
Reuse: {reference files or installed components}
Rules: pnpm check green; no new packages without asking; RLS in the same migration.
Done when (I will test on a phone with Juan's Cafe and Tala Apparel):
- {test 1}
- {test 2}
Finish: commit "{message}", push to dev, reply in 5 lines with what changed and how to test.
```